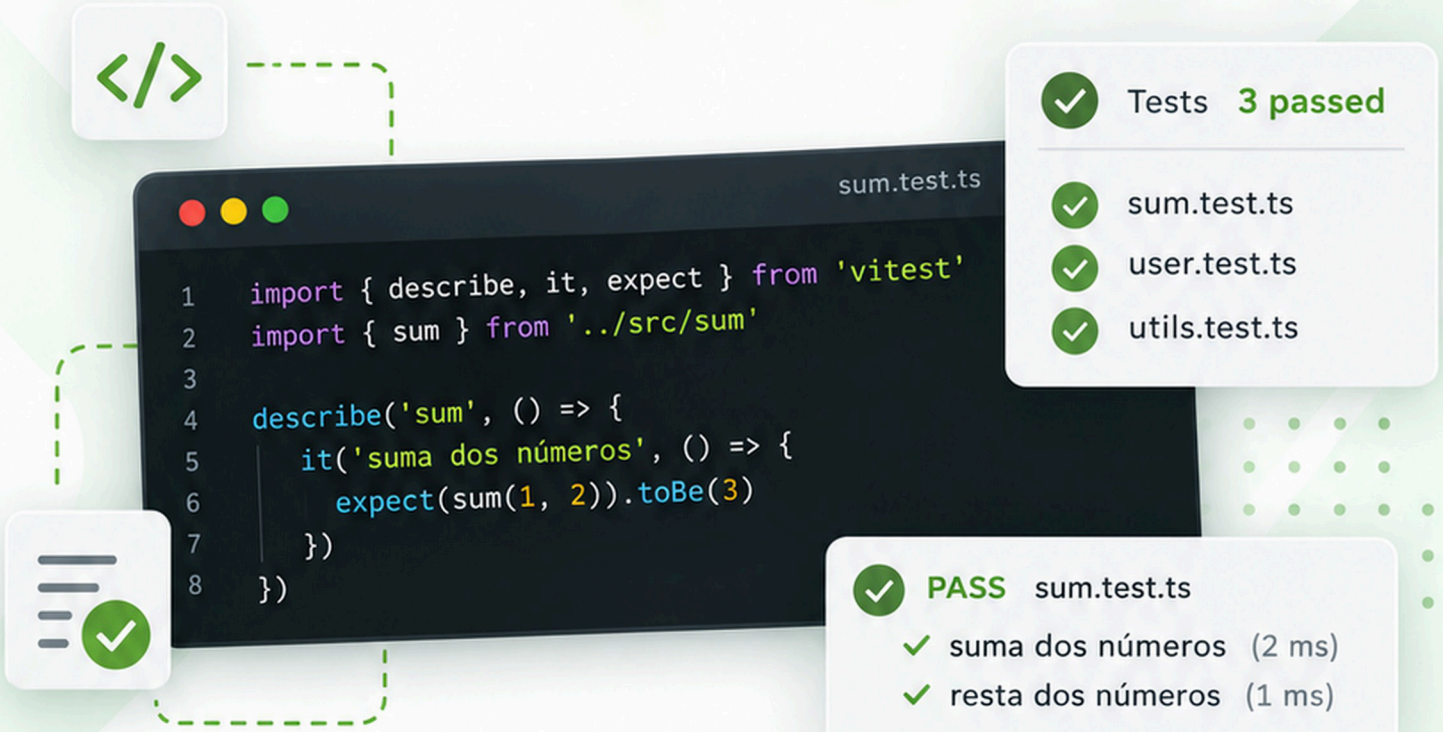
MANUAL DE

Vitest



AUTOR

G0nxI

ÍNDICE DE CONTENIDOS

Manual introductorio de Vitest

GUÍA DE LECTURA / DE LA INSTALACIÓN AL PRIMER TEST **Página**

1. ¿Qué es Vitest y para qué sirve? 3

1.1. ¿Qué es Vitest? 3

1.2. ¿Para qué sirve? 3

1.3. Origen, mantenimiento y versión actual 3

2. Conceptos clave de Vitest 3

2.1. ¿Qué son las pruebas automatizadas? 3

2.2. Estructura de una prueba 4

2.3. Funciones test() e it() 4

2.4. Comprobaciones con expect() 4

2.5. Agrupación de pruebas con describe() 4

3. Requisitos e instalación 4

3.1. Herramientas necesarias 4

3.2. Instalación de Node.js y npm 5

3.3. Instalación del editor de código 5

3.4. Creación e inicialización del proyecto 5

3.5. Instalación y configuración de Vitest 5

3.6. Estructura de carpetas y archivos 6

3.7. Comprobación de la instalación 6

4. Ejemplo práctico de uso 7

4.1. Presentación del ejemplo 7

4.2. Creación de la calculadora 7

4.3. Creación de las pruebas automatizadas 7

4.4. Ejecución de las pruebas 8

4.5. Detección de errores: una prueba fallida 9

4.6. Corrección de errores y resultado final 10

4.7. Posibles ampliaciones 10

5. Ventajas, inconvenientes y conclusión 11

5.1. Ventajas de Vitest 11

5.2. Inconvenientes de Vitest 11

5.3. Valoración personal y conclusión 11

6. Recursos y bibliografía 11

6.1. Documentación oficial 11

6.2. Otros recursos consultados 12

1. ¿Qué es Vitest y para qué sirve?

1.1. ¿Qué es Vitest?

Vitest permite comprobar automáticamente si las funciones de un programa hacen lo que esperamos. Por ejemplo, puede revisar que una calculadora sume dos números correctamente, sin tener que revisar el resultado a mano cada vez. Se integra con los proyectos que utilizan Vite. Vite ayuda a desarrollar y preparar aplicaciones web.

Ver también: 1.2. Para qué sirve · 3.5. Instalación

1.2. ¿Para qué sirve?

Sirve para detectar errores y comprobar que los cambios no han estropeado una función que antes funcionaba. Escribimos una prueba una vez y la volvemos a ejecutar cuando modificamos el programa. Si el resultado recibido no coincide con el esperado, Vitest señala el fallo.

El proyecto surgió para facilitar las pruebas en aplicaciones que usaban Vite. Con otras herramientas podía ser necesario mantener dos configuraciones distintas: una para desarrollar y otra para probar. Vitest aprovecha la configuración de Vite y permite trabajar con ambas tareas de una forma más sencilla.

Ver también: 4.1. Ejemplo práctico · 4.5. Prueba fallida

1.3. Origen, mantenimiento y versión actual

Vitest es un proyecto de código abierto. Lo mantiene un equipo internacional de desarrolladores, entre los que están Anthony Fu (uno de sus creadores) y Vladimir Sheremet (uno de sus principales mantenedores), junto con colaboradores de la comunidad. El repositorio principal contiene su código, la documentación y los ejemplos. Su licencia es MIT.

La versión utilizada en este manual es la 5.0.4.

Ver también: 6.1. Fuentes oficiales

2. Conceptos clave de Vitest

2.1. ¿Qué son las pruebas automatizadas?

Las pruebas automatizadas las ejecuta el ordenador. Si una prueba revisa una parte pequeña del programa, como una función, se llama prueba unitaria. Por ejemplo, puede comprobar que `sumar(2, 3)` devuelva `5`.

Si el resultado coincide con lo esperado, la prueba pasa. Si no coincide, falla. Cada caso debe tener un objetivo concreto y un resultado correcto conocido.

Ver también: 4.3. Pruebas automatizadas · 4.4. Ejecución

2.2. Estructura de una prueba

Una prueba suele tener tres pasos: preparar los datos, ejecutar la operación y revisar el resultado. Aquí preparamos 2 y 3, realizamos la suma y esperamos 5.

```
import { expect, test } from 'vitest';

test('suma dos valores', () => {
  const a = 2; // Preparar los datos.
  const b = 3;
  const resultado = a + b; // Ejecutar.
  expect(resultado).toBe(5); // Comprobar.
});
```

Ver también: 4.3. Pruebas automatizadas

2.3. Funciones test() e it()

`test()` define una prueba mediante un nombre y una función con sus comprobaciones. `it()` es un alias: hace lo mismo.

Ver también: 4.3. Pruebas automatizadas

2.4. Comprobaciones con expect()

`expect()` recibe el resultado que revisamos. `toBe()` indica el valor exacto esperado. La siguiente comprobación pasa porque 2 + 3 es 5.

```
expect(2 + 3).toBe(5);
```

Si utilizamos `2 - 3`, el resultado será -1 y la prueba fallará. El valor esperado debe representar el comportamiento correcto de la función.

Ver también: 4.5. Prueba fallida · 4.6. Corrección

2.5. Agrupación de pruebas con describe()

`describe()` reúne pruebas relacionadas bajo un nombre. Agruparemos la suma, la resta y la multiplicación en «Calculadora». Así es más fácil leer los resultados y localizar un fallo. El bloque de nuestra calculadora se muestra en el apartado 4.3.

3. Requisitos e instalación

3.1. Herramientas necesarias

Para hacer el ejemplo necesitas Node.js 24 LTS, npm, una terminal y un editor de código. Se puede trabajar en Windows o Linux.

Node.js ejecuta JavaScript y npm permite instalar Vitest. Necesitas Internet para las descargas. Los comandos están preparados para PowerShell en Windows. En Linux pueden variar; no he probado el proceso completo en ese sistema.

Ver también: 3.2. Node.js y npm · 3.3. Editor

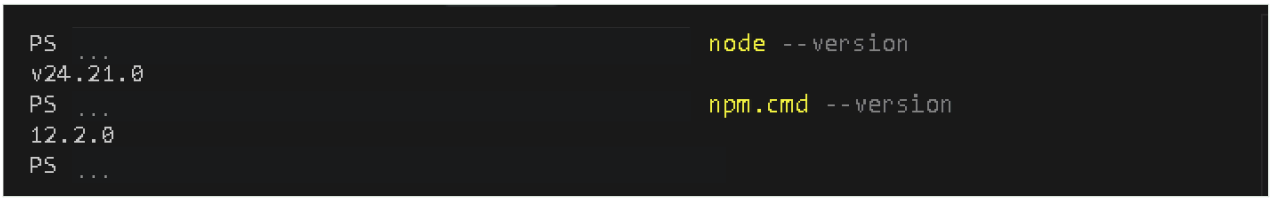
3.2. Instalación de Node.js y npm

Abre la página oficial de Node.js y elige la versión 24 LTS para tu sistema operativo. Sigue los pasos de instalación y, al terminar, abre una terminal nueva para consultar las versiones:

```
node --version
npm.cmd --version
```

Los dos comandos deben mostrar un número de versión. Si alguno no se reconoce, revisa la instalación y vuelve a abrir la terminal para que detecte el PATH actualizado.

01 · Node.js y npm G0nxI



Se comprobaron Node.js 24.21.0 y npm 12.2.0 desde PowerShell.

Ver también: 3.4. Crear el proyecto

3.3. Instalación del editor de código

Puedes usar el editor que prefieras; yo he utilizado Visual Studio Code. Cuando crees la carpeta `ejemplo`, ábrela desde «Archivo > Abrir carpeta». Después, entra en «Terminal > Nueva terminal» y comprueba que la ruta sea la de `ejemplo`.

Ver también: 3.4. Crear el proyecto · 4.2. Calculadora

3.4. Creación e inicialización del proyecto

En PowerShell, entra en la carpeta donde vayas a guardar el trabajo. Crea la carpeta principal del repositorio y, dentro, la carpeta del ejemplo:

```
mkdir manual-vitest-perez
cd manual-vitest-perez
mkdir ejemplo
cd ejemplo
npm.cmd init -y
```

`npm.cmd init -y` crea el archivo `package.json` con unos valores iniciales. Este archivo guarda los datos del proyecto, sus dependencias y los comandos que podremos ejecutar.

Ver también: 3.5. Instalar Vitest

3.5. Instalación y configuración de Vitest

Instala Vitest como dependencia de desarrollo y fija la versión usada en el manual. Después configura los comandos de pruebas y el tipo de módulos JavaScript:

```
npm.cmd install --save-dev --save-exact vitest@5.0.4
npm.cmd pkg set type=module
npm.cmd pkg set "scripts.test=vitest"
npm.cmd pkg set "scripts.test:run=vitest run"
```

`--save-dev` guarda Vitest entre las herramientas de desarrollo. `--save-exact` conserva la versión 5.0.4. El campo `type=module` permite utilizar `import` y `export`. El comando `test` activa el seguimiento de cambios y `test:run` realiza una ejecución y termina.

Ver también: 3.7. Comprobar la instalación · 4.3. Pruebas

3.6. Estructura de carpetas y archivos

Tras instalar las dependencias y crear los archivos del apartado 4, la carpeta `ejemplo` tendrá estos elementos:

Archivo o carpeta	Función
`package.json`	Información del proyecto, versión de Vitest y comandos de pruebas.
`package-lock.json`	Versiones resueltas de las dependencias; se genera al instalar.
`node_modules/`	Paquetes descargados por npm; no se suben al repositorio.
`calculadora.js`	Funciones de suma, resta y multiplicación.
`calculadora.test.js`	Casos de prueba que ejecutará Vitest.

Ver también: 4.2. Calculadora · 4.3. Pruebas

3.7. Comprobación de la instalación

Antes de escribir las pruebas, comprueba que la instalación de Vitest ha terminado y que puedes consultar su versión:

```
npx.cmd vitest --version
```

La salida debe identificar Vitest 5.0.4. Esta comprobación confirma que la herramienta está disponible en el proyecto. En el apartado 4 crearemos los archivos y ejecutaremos las pruebas.

02 · Versión de Vitest G0nxI



Se comprobó Vitest 5.0.4 desde la carpeta ejemplo.

Ver también: 4.4. Ejecutar las pruebas

4. Ejemplo práctico de uso

4.1. Presentación del ejemplo

El ejemplo consiste en una calculadora con tres funciones: sumar, restar y multiplicar. Las cinco pruebas se ejecutan desde la terminal. Comprobaremos resultados normales, una resta negativa y una multiplicación por cero.

Primero ejecutaremos las pruebas, después cambiaremos la suma para provocar un fallo y al final la corregiremos. Trabajaremos siempre dentro de `manual-vitest-perez/ejemplo`.

Ver también: 4.2. Calculadora · 4.3. Pruebas

4.2. Creación de la calculadora

En Visual Studio Code crea `calculadora.js` dentro de la carpeta `ejemplo` y copia el siguiente código:

calculadora.js

```
// Estas funciones reciben dos números y devuelven un resultado.
export function sumar(a, b) {
  return a + b;
}

export function restar(a, b) {
  return a - b;
}

export function multiplicar(a, b) {
  return a * b;
}
```

Cada función recibe dos números, `a` y `b`, y utiliza `return` para devolver el resultado. La palabra `export` permite importar las funciones desde el archivo de pruebas. Usaremos números enteros para centrarnos en las pruebas.

Ver también: 4.3. Pruebas automatizadas

4.3. Creación de las pruebas automatizadas

Crea `calculadora.test.js` en la misma carpeta. El nombre contiene `.test.js` para que Vitest lo reconozca como un archivo de pruebas:

calculadora.test.js

```
import { describe, expect, test } from 'vitest';
import { sumar, restar, multiplicar } from './calculadora.js';

// Agrupa los casos de prueba de nuestra calculadora.
describe('Calculadora', () => {
  test('suma dos números', () => {
    expect(sumar(2, 3)).toBe(5);
  });

  test('resta dos números', () => {
    expect(restar(8, 3)).toBe(5);
  });
```

calculadora.test.js (continuación)

```
  test('la resta puede dar un resultado negativo', () => {
    expect(restar(3, 8)).toBe(-5);
  });

  test('multiplica dos números', () => {
    expect(multiplicar(4, 3)).toBe(12);
  });

  test('multiplicar por cero devuelve cero', () => {
    expect(multiplicar(7, 0)).toBe(0);
  });
});
```

Importamos `describe`, `expect` y `test` de Vitest, junto con las funciones de la calculadora. El grupo `describe('Calculadora', ...)` reúne cinco casos independientes.

En cada caso, `expect()` recibe el resultado de una operación y `toBe()` indica el valor correcto. Además de las operaciones habituales, comprobamos que una resta pueda dar un número negativo y que multiplicar por cero devuelva cero.

Ver también: 2.3. test() e it() · 2.4. expect() · 2.5. describe()

4.4. Ejecución de las pruebas

Guarda los dos archivos y ejecuta este comando desde la carpeta `ejemplo`:

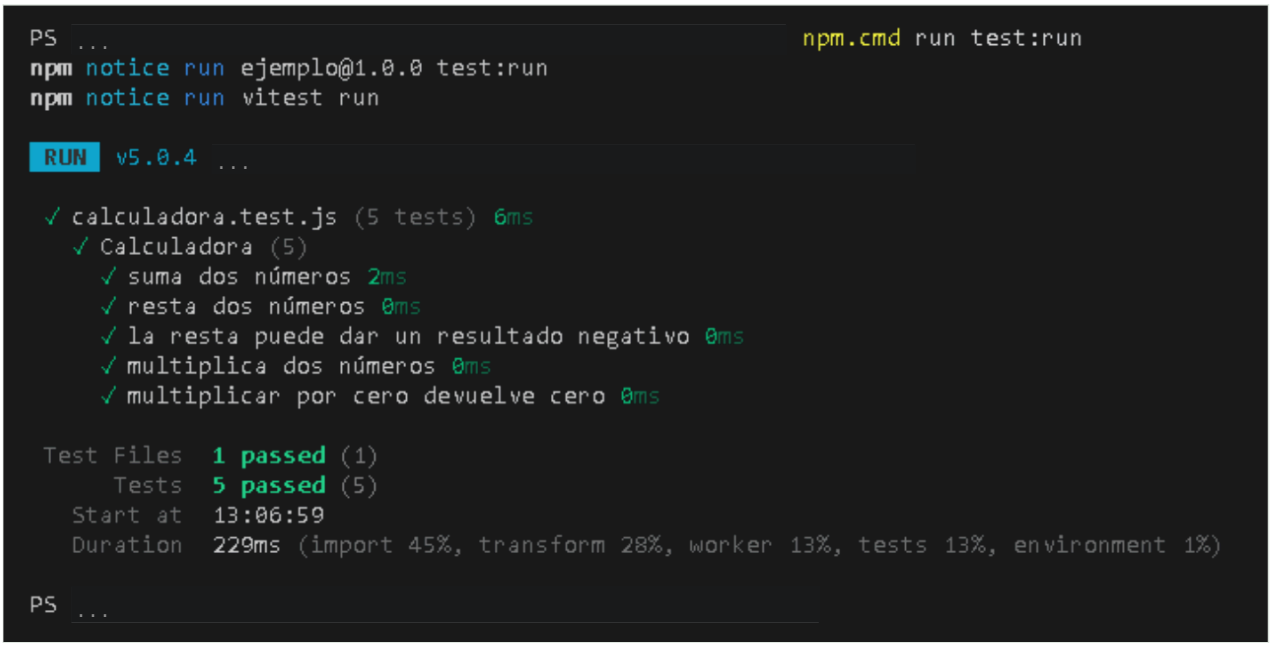
```
npm.cmd run test:run
```

Si todo está bien, Vitest mostrará un archivo y cinco pruebas aprobadas:

```
Test Files  1 passed (1)
     Tests  5 passed (5)
```

Los tiempos de ejecución pueden variar. Para repetir las pruebas automáticamente cuando guardes un cambio, utiliza `npm.cmd test`. En ese modo puedes pulsar `Ctrl+C` para detener Vitest.

03 · Primera ejecución G0nxI



Primera ejecución desde ejemplo: un archivo y cinco pruebas aprobadas.

Ver también: 4.5. Prueba fallida

4.5. Detección de errores: una prueba fallida

En `calculadora.js`, cambia la función `sumar` para que reste:

```
export function sumar(a, b) {
  return a - b; // Error introducido para la demostración.
}
```

En PowerShell, ejecuta:

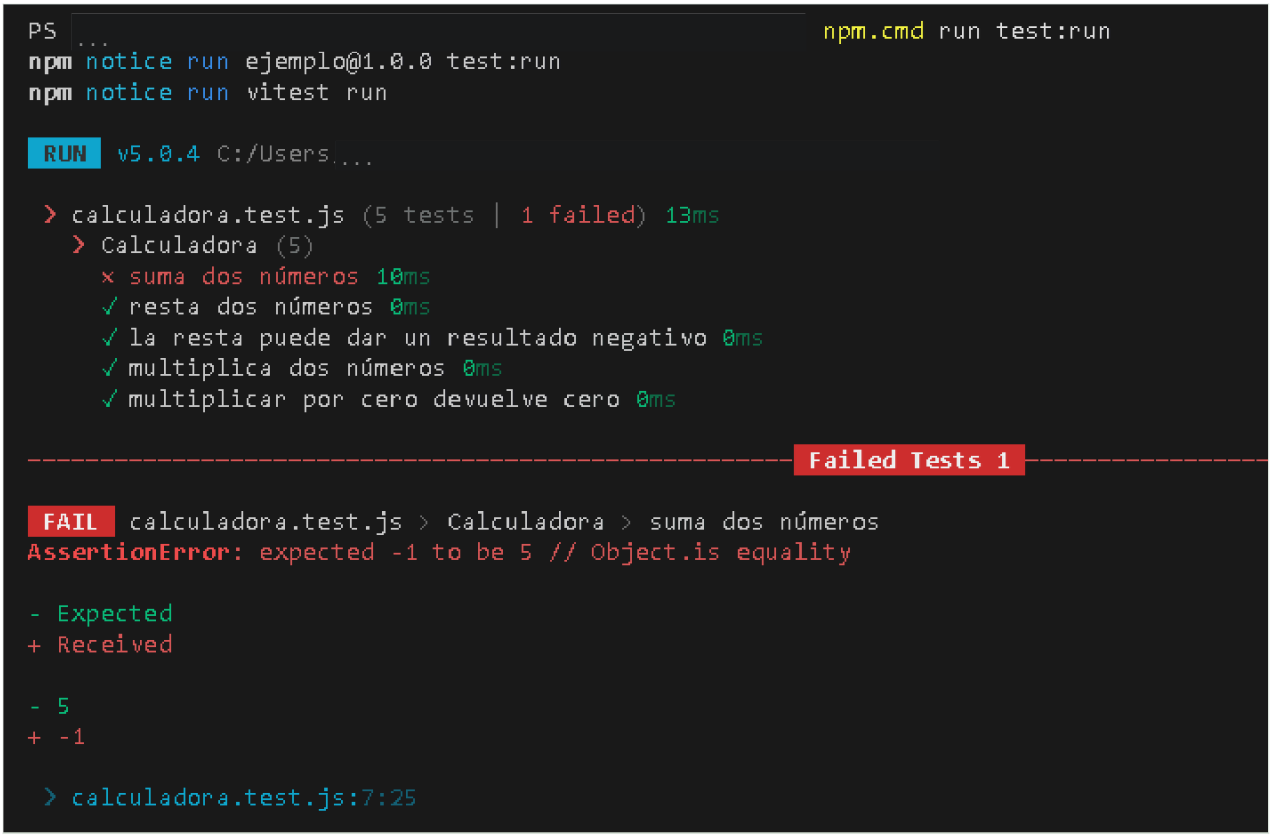
```
npm.cmd run test:run
```

La prueba «suma dos números» fallará, mientras que las otras cuatro seguirán pasando. Para los datos 2 y 3, la función devuelve -1, aunque la prueba espera 5. El mensaje principal indicará:

```
AssertionError: expected -1 to be 5
Test Files  1 failed (1)
     Tests  1 failed | 4 passed (5)
```

La prueba espera el resultado correcto: 5. El error está en la función sumar.

04 · Detección del error G0nxI



Tras cambiar la suma por una resta, una prueba falla y cuatro pasan. Se esperaba 5 y se recibió -1.

04.2 · Detalle de la prueba G0nxI



Vitest señala la comprobación de la suma en calculadora.test.js, línea 7.

Ver también: 4.6. Corregir el error · 2.4. expect()

4.6. Corrección de errores y resultado final

Devuelve la función a su versión correcta y guarda el archivo:

```
export function sumar(a, b) {
  return a + b;
}
```

En PowerShell, ejecuta:

```
npm.cmd run test:run
```

Las cinco pruebas vuelven a pasar al corregir la suma. El resultado esperado sigue siendo el mismo: hemos arreglado la función.

Este ciclo muestra para qué sirve Vitest en la práctica. Las mismas pruebas pueden avisarnos si un cambio posterior vuelve a estropear la suma, la resta o la multiplicación.

05 · Corrección y resultado G0nxI

```
1  // Estas funciones reciben dos números y devuelven un resultado.
2  export function sumar(a, b) {
3    return a + b; // Arreglado
4  }
```

```
PS ...                                                npm.cmd run test:run
npm notice run ejemplo@1.0.0 test:run
npm notice run vitest run

 RUN  v5.0.4 ...

 ✓ calculadora.test.js (5 tests) 5ms
   ✓ Calculadora (5)
     ✓ suma dos números 2ms
     ✓ resta dos números 0ms
     ✓ la resta puede dar un resultado negativo 0ms
     ✓ multiplica dos números 0ms
     ✓ multiplicar por cero devuelve cero 0ms

 Test Files  1 passed (1)
      Tests  5 passed (5)
   Start at  13:12:29
   Duration  247ms (transform 46%, import 34%, worker 11%, tests 9%)

PS ...
```

Se restauró return a + b; y se repitieron las pruebas: los cinco casos volvieron a aprobarse.

Ver también: 4.4. Ejecutar las pruebas

4.7. Posibles ampliaciones

- **Mocks**: simulan funciones o dependencias para controlar los datos de una prueba.
- **Cobertura de código**: muestra qué líneas, funciones y ramas se ejecutan. Una cobertura alta no garantiza que el programa sea correcto.
- **Pruebas en navegador**: permiten comprobar componentes e interacciones de una interfaz en un navegador real.

El catálogo oficial recoge también snapshots, tests asíncronos, hooks, filtros, paralelismo, varios proyectos, informes, interfaz visual, pruebas de tipos y benchmarks.

5. Ventajas, inconvenientes y conclusión

5.1. Ventajas de Vitest

- Repite las comprobaciones con un comando, sin revisar cada resultado a mano.
- Los nombres de las pruebas y los mensajes de error ayudan a localizar un fallo.
- Su integración con Vite facilita compartir la configuración de un proyecto web.
- Vuelve a ejecutar las pruebas al guardar los cambios.

Ver también: 4.4. Ejecución · 4.6. Corrección

5.2. Inconvenientes de Vitest

Preparar las pruebas y actualizarlas lleva tiempo. Hay que elegir bien qué vamos a probar: nuestros cinco casos no revisan todas las combinaciones ni entradas como texto o datos vacíos.

También hay que revisar los requisitos de Node.js cuando se cambia de versión de Vitest. Para probar interfaces y acciones de un navegador real se necesita preparar un entorno adicional; el ejemplo de este manual se centra en funciones JavaScript.

Ver también: 3.1. Herramientas · 4.7. Ampliaciones

5.3. Valoración personal y conclusión

Cuando cambié la suma por una resta, Vitest detectó el error. Después de corregirla, volvieron a pasar las cinco pruebas.

En el prototipo de tutorías usaría ese mismo proceso para comprobar reservas duplicadas y casos límite si continuáramos su desarrollo.

Ver también: 6.2. Recursos consultados

6. Recursos y bibliografía

6.1. Documentación oficial

Documentación y recursos oficiales consultados.

Recurso y enlace	Descripción
Primeros pasos Guía de primeros pasos	Instalación, requisitos y ejecución de la primera prueba.
Por qué Vitest Origen y objetivos de Vitest	Explica el origen del proyecto y el problema que intenta resolver.
test() e it() Referencia de test() e it()	Referencia para declarar pruebas y configurar sus opciones.
describe() Referencia de describe()	Explica cómo agrupar varias pruebas relacionadas.
expect() Referencia de expect()	Describe las comprobaciones y los resultados esperados.

Recurso y enlace	Descripción
Configuración y ampliaciones Opciones de configuración	Opciones de configuración. Ampliaciones: mocks · cobertura · navegador · funciones.
Ejemplos oficiales Catálogo de ejemplos	Índice de ejemplos con código y proyectos para probar en línea.
Repositorio y desarrolladores Código fuente en GitHub	Código y equipo de Vitest. Fuentes de los desarrolladores: perfiles de Anthony Fu y Vladimir Sheremet; equipo de VoidZero.
Versiones publicadas Historial de versiones	Historial de versiones y notas de los cambios.

6.2. Otros recursos consultados

Vitest: Web oficial de Vitest — Web principal del proyecto y acceso a su documentación.

Node.js: Descargar Node.js — Descarga oficial de Node.js para preparar el entorno de pruebas.

He utilizado estas herramientas de IA como apoyo para comprender conceptos y revisar la redacción. Después ejecuté las pruebas en mi equipo para comprobar el ejemplo:

- ChatGPT (OpenAI): Web de ChatGPT
- Gemini (Google): Web de Gemini

Repositorio: https://github.com/G0nxI/manual-vitest-perez